Helix OTA

Helix OTA — Delta Updates Design Document

> **Document ID:** `HELOTA-DELTA-001` **Version:** 1.0.2
> **Status:** Active **Last Updated:** 2026-03-05
> **Constitution Reference:** HelixConstitution v1 §1–§4
> **Target Platform:** Android 15 on Orange Pi 5 Max (RK3588) **Precedes:** 1.0.1-Rollback Support Design (HELOTA-ROLLBACK-001) **Depends On:** System Architecture v1.0.0 (HELOTA-ARCH-001), REST API Specification v1.0.0, Database Schema v1.0.0

Table of Contents

---

1. Delta Updates Overview

1.1 What Are Delta Updates and Why They Matter

A delta update (also called a differential or incremental update) is an OTA package that contains only the binary differences between the device's currently installed system image (the source) and the target system image, rather than the complete target image. Instead of downloading a full 2 GB OTA package to update from version 15.0.0 to 15.0.1, a device may download a 200 MB delta package that encodes only the changed blocks, achieving a 90% bandwidth reduction.

The impact of delta updates on fleet operations is substantial and multi-dimensional:

Bandwidth Reduction (60–90%): The primary motivation. For a fleet of 10,000 Orange Pi 5 Max devices, a single full OTA update of 2 GB per device consumes 20 TB of egress bandwidth. A delta update at 80% reduction drops this to 4 TB — saving 16 TB per update cycle. At AWS S3 egress pricing ($0.09/GB), this translates to a cost reduction of approximately $1,440 per update cycle for a 10,000-device fleet.

Update Duration Reduction: Smaller payloads download faster and apply faster. A 2 GB full OTA over a 50 Mbps connection takes approximately 5.3 minutes to download. A 200 MB delta takes 32 seconds. This reduces the update window and the time a device spends in the vulnerable "updating" state.

Device Reliability: Shorter download and apply windows mean fewer opportunities for network interruptions, power failures, or storage errors to corrupt the update. The probability of a failure during a 30-second delta apply is significantly lower than during a 5-minute full update apply.

Server Infrastructure Load: With delta updates, artifact storage, CDN caching, and download throughput requirements are dramatically reduced. A server serving 10,000 delta downloads at 200 MB each handles 2 TB, compared to 20 TB for full updates.

Rollout Velocity: Because delta updates complete faster, the rollout engine can evaluate cohort health more quickly and advance to the next percentage sooner. A rollout that previously took 48 hours to reach 100% of the fleet might complete in 12 hours with delta updates.

1.2 Delta vs. Full Update Trade-offs

Delta updates are not universally superior to full updates. The design must account for several trade-offs:

Property	Full Update	Delta Update
Payload Size	Complete target image (1–3 GB)	Binary differences only (100–800 MB)
Source Requirement	None — works from any state	Requires exact source partition state
Generation Complexity	Simple — package target image	Complex — requires source + target diff
Generation Time	Minutes	Hours (2–8h for large partitions)
Apply Time	Linear in payload size	Linear in changed blocks + source reads
Failure Mode	Self-contained; no source dependency	Source partition must match expected hash
Storage Overhead	One artifact per version	N×M delta artifacts for N source × M target
Security Model	Single payload verification	Source hash verification + delta verification
Recovery	Re-download full package	Fall back to full update on any failure

The critical constraint is that **delta updates require the source partition to be in an exact, verified state**. If the device's source partition has been modified (e.g., by an unofficial root modification, a filesystem repair, or a partial previous update), the delta apply will fail because the binary diff was computed against a specific source image. This is why source partition hash verification is a mandatory step before delta application (see Section 6.3).

1.3 Delta Update Strategies

Three fundamental strategies exist for generating delta payloads, each with different characteristics:

Binary Diff (bsdiff/puffdiff): The most fine-grained strategy. Computes the byte-level difference between the source and target partition images using algorithms like bsdiff (for general binary data) or puffdiff (optimized for deflated streams). Binary diff produces the smallest delta payloads but is computationally expensive to generate and requires the entire source image to be available during application. AOSP's `update_engine` uses this strategy via SOURCE_BSDIFF and PUFFDIFF operations in the payload.

Block-Level: Operates at the block device level (typically 4 KB blocks). For each block in the target image, the algorithm checks whether the same block exists in the source image (by hash comparison). Matching blocks are encoded as SOURCE_COPY operations (zero-cost); changed blocks are either included verbatim (REPLACE) or compressed as a binary diff (SOURCE_BSDIFF). Block-level delta is the strategy used by AOSP's `brillo_update_payload` tool and provides excellent performance for typical system updates where many blocks are unchanged between minor versions.

File-Level: Operates at the filesystem level by comparing individual files between source and target. Unchanged files are skipped, and changed files are included in the delta payload. File-level deltas are simpler but less efficient than block-level deltas because they cannot handle partial file changes — a 1-byte change in a 100 MB file requires the entire 100 MB file to be included. This strategy is more applicable to Linux package-based updates (deb/rpm) than to Android partition-based updates.

Helix OTA 1.0.2 uses block-level delta with binary diff compression, matching the AOSP `ota_from_target_files` incremental OTA approach. This is the optimal strategy for Android A/B partition updates because: (1) it produces the smallest payloads by leveraging both unchanged block copies and compressed binary diffs, (2) it is natively supported by `update_engine`'s SOURCE_COPY, SOURCE_BSDIFF, and PUFFDIFF operations, and (3) the AOSP tooling is mature and well-tested across billions of Android devices.

---

2. Android OTA Delta Mechanism

2.1 How AOSP Generates Delta OTA Packages (Incremental OTA)

AOSP generates delta OTA packages using the `ota_from_target_files` tool with the `--incremental_from` flag. The tool takes two `target_files.zip` packages as input — one representing the source version and one representing the target version — and produces an incremental OTA zip that contains only the differences.

The `target_files.zip` is an intermediate build artifact produced by the Android build system. It contains:

- `IMAGES/system.img` — The complete system partition image
- `IMAGES/vendor.img` — The complete vendor partition image
- `IMAGES/boot.img` — The boot image (kernel + ramdisk)
- `IMAGES/product.img`, `IMAGES/odm.img` — Additional partition images
- `META/` — Metadata including file system manifests, signing keys, and build info

The generation process works as follows:

1. **Extract partition images** from both source and target `target_files.zip` files.
2. **Compute block-level differences** for each partition. For each 4 KB block in the target image:
   - Compute the SHA-256 hash of the block.
   - Compare against all source blocks. If a matching hash is found, encode as SOURCE_COPY referencing the source block offset.
   - If no match, try SOURCE_BSDIFF: compute a binary diff between the source block neighborhood and the target block, and include the compressed patch if it is smaller than the raw block data.
   - If binary diff is not smaller, encode as REPLACE (raw block data) or REPLACE_BZ (bzip2-compressed block data).
3. **For deflated partitions** (system, vendor on Android 15), apply PUFFDIFF: decompress both source and target, compute the diff on the decompressed data, and re-compress. This produces significantly smaller deltas for partitions that contain compressed content.
4. **Package the delta operations** into a `payload.bin` protobuf following the `chromeos_update_engine` DeltaArchiveManifest format.

5. **Generate `payload_properties.txt`** containing the payload metadata (size, SHA-256 hash, metadata offset).
6. **Generate `care_map.pb`** for dm-verity care map.
7. **Sign the OTA package** with the release key.
8. **Package** everything into a ZIP file with the standard OTA structure.

2.2 payload.bin Delta Operations

The `payload.bin` file inside an OTA zip is a protobuf-encoded `DeltaArchiveManifest` that describes a sequence of operations for each partition. For delta (incremental) OTAs, the manifest contains three key operation types:

SOURCE_COPY: Instructs `update_engine` to copy one or more blocks from the source partition to the target partition. This is the most efficient delta operation — it requires zero download bandwidth for the affected blocks and minimal CPU time. The operation specifies source block ranges and destination block ranges. Example: “Copy blocks 0–99 from source system partition to blocks 0–99 of target system partition.” This is used when the block content is identical between source and target.

SOURCE_BSDIFF: Instructs `update_engine` to apply a bsdiff patch to source partition blocks to produce target partition blocks. The delta payload includes the compressed bsdiff patch data. The operation specifies: source block ranges to read, the patch data offset and length within the payload, and the destination block ranges. Example: “Read blocks 100–149 from source, apply bsdiff patch at offset 0x5000 (length 0x2000), write result to blocks 100–149 of target.” This is used when blocks have changed but are similar enough that binary diff produces a smaller patch than the raw data.

PUFFDIFF: An optimized diff algorithm for deflated (gzip-compressed) data. Android 15 partitions (system, vendor) contain many files that are individually compressed with deflate. A naive binary diff on compressed data produces large patches because small changes in uncompressed content cause large changes in the compressed byte stream. Puffdiff decompresses both source and target data, computes the diff on the decompressed streams, and re-encodes the result. This typically produces patches 30–50% smaller than SOURCE_BSDIFF for partitions containing compressed content.

The protobuf manifest also includes:

- `source_partition_hash` — SHA-256 hash of each source partition, used for pre-apply verification

- `new_partition_info` — SHA-256 hash and size of each target partition, used for post-apply verification
- `partition_operations` — The list of operations (SOURCE_COPY, SOURCE_BSDIFF, PUFFDIFF, REPLACE, REPLACE_BZ) for each partition
- `minor_version` — Delta format version (currently 6 for AOSP)

2.3 How update_engine Applies Delta Operations

The Android `update_engine` daemon applies delta operations in a specific sequence:

1. **Source Partition Verification:** Before applying any delta operation, `update_engine` computes the SHA-256 hash of each source partition (the inactive slot's partitions) and compares it against the `source_partition_hash` in the manifest. If any hash does not match, the delta application is aborted with error `DELTA_PRECHECK_FAILED`. This ensures the source state matches what the delta was generated against.

2. **Operation Application:** For each partition, `update_engine` processes the operations sequentially:

   - SOURCE_COPY: Read blocks from source partition, write to target partition (inactive slot).
   - SOURCE_BSDIFF: Read source blocks, apply bsdiff patch from payload, write result to target.
   - PUFFDIFF: Read source blocks, decompress, apply puffdiff patch from payload, recompress, write result.
   - REPLACE / REPLACE_BZ: Write raw/compressed data from payload directly to target.

3. **Post-Apply Verification:** After all operations for a partition are complete, `update_engine` computes the SHA-256 hash of the target partition and compares it against `new_partition_info.hash`. If the hash does not match, the update is failed with `POST_INSTALL_VERIFICATION_FAILED`.

4. **Post-Install Script Execution:** If the manifest includes `postinstall` scripts, they are executed in a chroot environment on the newly-written partition.

5. **Boot Slot Switch:** After all partitions are verified, `update_engine` calls `setActiveBootSlot(newSlot)` and reports success to the Helix OTA client via the Binder IPC callback.

2.4 Source Partition Verification for Delta Updates

Source partition verification is the single most critical safety check in the delta update flow. Without it, applying a delta to a source partition that differs from the expected state would produce a corrupted target partition, potentially bricking the device.

The verification process:

1. The delta payload manifest includes a `source_partition_hash` for each partition (system, vendor, boot, etc.).
2. Before applying any delta operation, `update_engine` reads the entire source partition from the inactive slot, computes its SHA-256 hash, and compares it against the manifest's expected hash.
3. If the hashes match, the delta is applied.
4. If any hash does not match, `update_engine` aborts with error code `kErrorCodeDownloadPartitionHashMismatch` (error code 28 in AOSP). The Helix OTA client catches this error and initiates the fallback to a full update (see Section 6.4).

Common causes of source partition hash mismatch:

- The device was previously updated with a modified OTA package not in the Helix OTA system (e.g., a sideloaded update).
- The source partition was corrupted by a filesystem error, storage media degradation, or an interrupted previous update.
- The device's `current_version` in the Helix OTA registry does not reflect the actual partition state (e.g., a rollback occurred but was not reported).

2.5 Delta Generation from target_files

For Helix OTA's server-side delta generation, the input is not `target_files.zip` (which is a build system artifact) but rather the completed full OTA zip packages. The delta generation pipeline extracts partition images from the full OTA zips and then runs the delta computation:

1. **Extract payload.bin** from both source and target OTA zips.
2. **Extract partition images** from each `payload.bin` using `extract_partition_images` (a custom tool that reads the DeltaArchiveManifest and applies the operations to produce raw partition images).

3. **Run `ota_from_target_files --incremental_from`** with the extracted source and target images to produce the incremental OTA package.
4. **Validate** the delta package by applying it to a reference source image and comparing the result with the target image.

This two-step process (extract, then diff) is necessary because the Helix OTA server does not have access to the build system's `target_files.zip` — it only has the published OTA zip packages.

---

3. Delta Generation Service

3.1 Server-Side Delta Artifact Generation

The delta generation service runs server-side as a background worker, triggered automatically when a new artifact is uploaded. It produces delta OTA packages for all eligible source versions targeting the newly-uploaded artifact.

The service is designed with the following principles:

- **Asynchronous Generation:** Delta generation is a time-consuming operation (2–8 hours for large partitions). It must not block the artifact upload flow.
- **Idempotent:** Re-running generation for the same source→target pair produces the same output. Failed generations can be safely retried.
- **Resource-Bounded:** Generation jobs run in a worker pool with configurable concurrency limits and memory caps to prevent resource exhaustion.
- **Observable:** Every generation job reports progress, timing, and size metrics for monitoring and alerting.

3.2 Input and Output

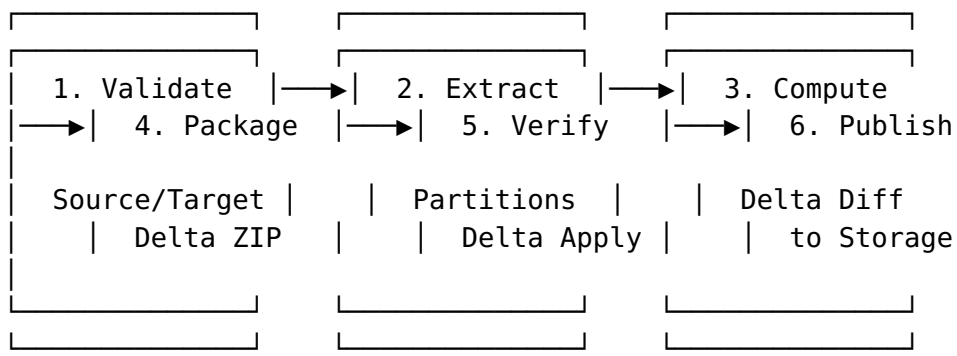
Input: Two OTA zip files — the source artifact (representing the device's current version) and the target artifact (the version the device will update to).

Output: A delta OTA zip file containing an incremental `payload.bin` with SOURCE_COPY, SOURCE_BSDIFF, and PUFFDIFF operations, plus `payload_properties.txt`, `care_map.pb`, and the standard OTA metadata.

The output delta zip is stored alongside the full target artifact in S3/MinIO, indexed in the `delta_artifacts` database table (see Section 5.5), and made available to devices via the update check API.

3.3 Generation Pipeline

The delta generation pipeline consists of six stages:



Stage 1 — Validate Source/Target: Verify that both source and target artifacts exist, are in `VALID` status, and have compatible hardware models. Verify that source version < target version. Reject cross-OS-type deltas (e.g., Android→Linux).

Stage 2 — Extract Partitions: Download both full OTA zips from S3/MinIO to temporary local storage. Extract `payload.bin` from each. Parse the `DeltaArchiveManifest` protobuf to identify partition names and extract raw partition images. On the Orange Pi 5 Max (RK3588), the relevant partitions are: `system`, `vendor`, `boot`, `product`, `odm`.

Stage 3 — Compute Delta Diff: For each partition that exists in both source and target, compute the block-level delta using `brillo_update_payload` / `ota_from_target_files --incremental_from`. The tool produces a `payload.bin` containing the delta operations. Partitions that are unchanged between source and target produce SOURCE_COPY-only operations (zero-cost delta).

Stage 4 — Package Delta ZIP: Assemble the delta `payload.bin`, `payload_properties.txt`, `care_map.pb`, and `META-INF/` directory into a standard OTA zip structure. Compute SHA-256 hash of the delta zip for integrity verification.

Stage 5 — Verify Delta Apply: Apply the delta to the extracted source partition images and compare the result with the target partition images. This verification step

ensures the delta is correct and can be safely deployed. If verification fails, the generation is marked as `FAILED` and the delta is not published.

Stage 6 — Publish to Storage: Upload the verified delta zip to S3/MinIO at path `deltas/{source_artifact_id}/{target_artifact_id}/delta_ota.zip`. Create a database record in `delta_artifacts`. Publish `delta.generated` event to EventBus.

3.4 Resource Requirements

Delta generation is resource-intensive. The following resource allocations are required per generation job:

Resource	Minimum	Recommended	Notes
CPU	4 cores	8 cores	bsdiff and puffdiff are CPU-intensive; parallelize by partition
RAM	16 GB	32 GB	bsdiff requires ~8× the partition size in RAM; puffdiff requires ~4×
Disk	20 GB temp	40 GB temp	Source + target images + delta output + verification
Time	2 hours	4 hours	Depends on partition sizes and number of changed blocks

For the Orange Pi 5 Max with ~4 GB per slot of partition data, a single delta generation job requires approximately 32 GB of RAM and 40 GB of temporary disk space. The generation server should be a dedicated high-memory instance (e.g., AWS r6i.2xlarge with 64 GB RAM).

3.5 Using ota_from_target_files for Delta Generation

The AOSP `ota_from_target_files` tool is the standard mechanism for generating incremental OTA packages. For Helix OTA's server-side generation, we wrap this tool in a containerized environment:

```
#!/bin/bash
# generate_delta_ota.sh — Generate incremental OTA from two
        full OTA zips
# Usage: generate_delta_ota.sh <source_ota.zip>
        <target_ota.zip> <output.zip> <key_path>

set -euo pipefail

SOURCE_OTA="$1"
TARGET_OTA="$2"
OUTPUT_ZIP="$3"
SIGNING_KEY="$4"

WORK_DIR=$(mktemp -d /tmp/helix_delta_XXXXXX)
trap "rm -rf ${WORK_DIR}" EXIT

echo "[helix-delta-gen] Starting delta generation"
echo "[helix-delta-gen] Source: ${SOURCE_OTA}"
echo "[helix-delta-gen] Target: ${TARGET_OTA}"
echo "[helix-delta-gen] Output: ${OUTPUT_ZIP}"
echo "[helix-delta-gen] Work dir: ${WORK_DIR}"

# Step 1: Extract payload.bin from source OTA
echo "[helix-delta-gen] Extracting source payload..."
mkdir -p "${WORK_DIR}/source"
unzip -q -o "${SOURCE_OTA}" "payload.bin"
        "payload_properties.txt" \
    -d "${WORK_DIR}/source/"

# Step 2: Extract payload.bin from target OTA
echo "[helix-delta-gen] Extracting target payload..."
mkdir -p "${WORK_DIR}/target"
unzip -q -o "${TARGET_OTA}" "payload.bin"
        "payload_properties.txt" \
    -d "${WORK_DIR}/target/"

# Step 3: Extract partition images from source payload
```

```
echo "[helix-delta-gen] Extracting source partition
        images..."
python3 /opt/helix/extract_images.py \
    --payload "${WORK_DIR}/source/payload.bin" \
    --output-dir "${WORK_DIR}/source_images/" \
    --partitions system vendor boot product odm

# Step 4: Extract partition images from target payload
echo "[helix-delta-gen] Extracting target partition
        images..."
python3 /opt/helix/extract_images.py \
    --payload "${WORK_DIR}/target/payload.bin" \
    --output-dir "${WORK_DIR}/target_images/" \
    --partitions system vendor boot product odm

# Step 5: Create target_files-like structure for
        ota_from_target_files
echo "[helix-delta-gen] Creating target_files structures..."
python3 /opt/helix/build_target_files.py \
    --source-images "${WORK_DIR}/source_images/" \
    --target-images "${WORK_DIR}/target_images/" \
    --source-ota "${SOURCE_OTA}" \
    --target-ota "${TARGET_OTA}" \
    --output-dir "${WORK_DIR}/target_files/"

# Step 6: Generate incremental OTA
echo "[helix-delta-gen] Computing delta (this may take
        several hours)..."
ota_from_target_files \
    --incremental_from "${WORK_DIR}/target_files/source/" \
    -k "${SIGNING_KEY}" \
    "${WORK_DIR}/target_files/target/" \
    "${OUTPUT_ZIP}"

# Step 7: Verify the generated delta
echo "[helix-delta-gen] Verifying delta package..."
python3 /opt/helix/verify_delta.py \
    --delta "${OUTPUT_ZIP}" \
    --source-images "${WORK_DIR}/source_images/" \
    --target-images "${WORK_DIR}/target_images/"

DELTA_SIZE=$(stat -c%s "${OUTPUT_ZIP}")
SOURCE_SIZE=$(stat -c%s "${SOURCE_OTA}")
SAVINGS=$((100 - (DELTA_SIZE * 100 / SOURCE_SIZE)))

echo "[helix-delta-gen] Delta generation complete"
echo "[helix-delta-gen] Delta size: $(numfmt --to=iec $
        {DELTA_SIZE})"
echo "[helix-delta-gen] Source size: $(numfmt --to=iec $
        {SOURCE_SIZE})"
echo "[helix-delta-gen] Bandwidth savings: ${SAVINGS}%"
```

3.6 Custom Delta Generator for Non-Android Platforms

While the initial 1.0.2 release targets Android's incremental OTA format, the `helix-delta-gen` submodule is designed for extensibility. Future versions will support:

- **Linux rootfs delta:** Using bsdiff on raw partition images for A/B rootfs updates (1.1.0).
- **OSTree static deltas:** Generating OSTree static delta files for rpm-ostree and Flatpak-based systems (1.1.0).
- **Windows delta:** Using MSIX delta packages or custom binary diff for Windows service updates (1.2.0).

The extensibility is achieved through a `DeltaGenerator` interface (see Section 7) that abstracts the platform-specific generation logic behind a common API.

---

4. Delta Selection Logic

4.1 Determining Delta Availability During Update Check

When a device performs an update check, the server must determine whether a delta update is available for that specific device. The selection logic is:

1. **Identify device's source version:** The device reports its `current_version` in the update check request.
2. **Find compatible delta:** Query the `delta_artifacts` table for a delta where `source_artifact_id` matches the device's current version's artifact and `target_artifact_id` matches the rollout's target artifact.
3. **Validate delta status:** The delta must have `status = 'GENERATED'` (not `PENDING`, `GENERATING`, or `FAILED`).
4. **Validate hardware compatibility:** The delta's hardware model must match the device's hardware model.
5. **Return delta or full update:** If a valid delta is found, return the delta artifact information. If not, return the full update artifact information.

4.2 Source Version Matching

The source version must match exactly. This is a strict requirement because delta updates are computed against a specific source partition image — a delta generated from

version 15.0.0 to 15.0.1 will not work if the device's actual source partition has a different hash than the 15.0.0 image used during generation.

The matching is performed by comparing the `version` field of the source artifact with the device's `current_version`. If they do not match exactly (string equality), the delta is not eligible.

4.3 Fallback to Full Update

If no delta is available for a device's source→target version pair, the server falls back to returning the full update. The device receives the same `UpdateInfo` response but with `update_type = "FULL"` instead of `"DELTA"`. The client handles both types identically from a download/verify/apply perspective — the difference is transparent to the update flow.

Reasons for falling back to a full update:

- No delta has been generated for this source→target pair (new source version, or generation failed).
- The delta generation is still in progress (`status = 'GENERATING'`).
- The delta was generated but failed verification (`status = 'FAILED'`).
- The device's `current_version` does not match any known source artifact.
- The delta was generated for a different hardware model than the device's.

4.4 Delta Chain Length Limits

A delta chain is a sequence of delta updates: A→B, then B→C, then C→D. While each individual delta is small, a device that is many versions behind must apply each delta sequentially, which increases total update time and the risk of a failure at any step.

Helix OTA enforces a **maximum delta chain length of 3**. If a device is more than 3 versions behind the target, the server returns a full update instead of requiring the device to apply 4+ sequential deltas.

The chain length is computed as the number of sequential version jumps from the device's `current_version` to the target version, based on the version graph stored in the `delta_artifacts` table. For example:

- Device on 15.0.0, target 15.0.1: Chain length 1 → delta OK
- Device on 15.0.0, target 15.0.2 (no direct 15.0.0→15.0.2 delta, but 15.0.0→15.0.1 and 15.0.1→15.0.2 exist): Chain length 2 → delta OK
- Device on 15.0.0, target 15.0.3 (three-step chain): Chain length 3 → delta OK
- Device on 15.0.0, target 15.0.4 (four-step chain): Chain length 4 → full update returned

In practice, most deployments generate direct deltas from the last N source versions to the latest target version, avoiding the need for multi-step chains entirely. The chain limit is a safety net for unusual cases.

---

5. Server-Side Delta Management

5.1 Delta Artifact Storage and Indexing

Delta artifacts are stored in S3/MinIO alongside full artifacts, using a distinct key prefix:

```
artifacts/{artifact_id}/{filename}                    ← Full OTA zip
deltas/{source_artifact_id}/{target_artifact_id}/delta_ota.zip  ← Delta OTA zip
```

This naming convention ensures: (1) deltas are logically separated from full artifacts, (2) the source→target relationship is encoded in the key, (3) listing all deltas for a given source or target version is efficient via S3 prefix listing.

5.2 Delta Compatibility Matrix

The delta compatibility matrix is a logical construct derived from the `delta_artifacts` database table. It answers the question: "Given a device running source version X, which target versions have a delta available?"

Example compatibility matrix for an RK3588 fleet:

Source Version	Target Version	Delta Size	Full Size	Savings	Status
15.0.0	15.0.1	245 MB	2.1 GB	88%	GENERATED
15.0.0	15.0.2	310 MB	2.1 GB	85%	GENERATED
15.0.1	15.0.2	180 MB	2.1 GB	91%	GENERATED
15.0.2	15.1.0	620 MB	2.3 GB	73%	GENERATED
14.9.0	15.0.2	—	—	—	NOT_GENERATED (source too old)

The matrix is computed on-demand by the `DeltaService` when processing update check requests. It is cached in Redis with a TTL of 300 seconds.

5.3 Automatic Delta Generation on New Artifact Upload

When a new full artifact is uploaded and passes validation, the server automatically triggers delta generation for the N most recent source versions (configurable, default: 3). This is implemented via the EventBus:

1. `ArtifactService.UploadArtifact` completes successfully and publishes `artifact.uploaded` event.
2. `DeltaService` subscribes to `artifact.uploaded` events.
3. On receiving the event, `DeltaService` queries the `artifacts` table for the N most recent artifacts with the same `os_type` and overlapping `hardware_compatibility`.
4. For each eligible source artifact, `DeltaService` creates a `delta_artifacts` record with `status = 'PENDING'` and enqueues a generation job.
5. The generation worker picks up the job, updates status to `GENERATING`, runs the pipeline, and updates status to `GENERATED` or `FAILED`.

This automatic generation ensures that deltas are available by the time devices begin checking for the new update. For a typical minor version update (e.g., 15.0.1 → 15.0.2), the delta generation completes within 2–4 hours of the artifact upload — well before the rollout percentage reaches even 5%.

5.4 Delta Artifact Cleanup Policy

Delta artifacts accumulate over time. Without cleanup, a fleet with 20 version releases would have up to 20×19 = 380 delta artifacts (N×(N-1) combinations). To prevent unbounded storage growth, the following cleanup policy is enforced:

1. **Maximum source age:** Deltas where the source artifact is older than 90 days are automatically deleted. Devices running versions older than 90 days receive full updates.
2. **Maximum delta count per target:** A maximum of 5 delta source versions are retained per target version. When a 6th is generated, the oldest is deleted.
3. **Failed deltas:** Delta artifacts with `status = 'FAILED'` and `updated_at` older than 7 days are automatically cleaned up.
4. **Orphaned deltas:** Deltas where either the source or target artifact has been deleted are automatically removed.

The cleanup job runs daily as a scheduled background task. It publishes `delta.cleaned_up` events for observability.

5.5 Database Schema Additions for Delta Tracking

See Section 11 for the complete SQL schema for the `delta_artifacts` table and associated indexes.

---

6. Client-Side Delta Handling

6.1 Update Check Response with Delta Availability

The existing `POST /api/v1/updates/check` endpoint is extended to include delta availability information in the response. When a delta is available for the device's source version, the response includes:

```
{
  "update_available": true,
  "artifact_id": "art_01HTARGET",
  "update_type": "DELTA",
  "source_version": "15.0.0",
  "target_version": "15.0.2",
  "download_url": "/api/v1/artifacts/dlt_01HDELTA01/
        download",
```

```
  "sha256": "a1b2c3d4...",
  "size_bytes": 245000000,
  "size_savings_percent": 88,
  "full_update_fallback": {
    "artifact_id": "art_01HTARGET",
    "download_url": "/api/v1/artifacts/art_01HTARGET/
        download",
    "sha256": "e5f6a7b8...",
    "size_bytes": 2100000000
  },
  "metadata": {
    "mandatory": false,
    "deadline": "2026-04-15T00:00:00Z"
  }
}
```

The `full_update_fallback` object is always included when `update_type = "DELTA"`, allowing the client to fall back without making a second update check request.

6.2 Delta Download and Verification

The delta download and verification flow is identical to the full update flow from the client's perspective:

1. **Download:** The `DownloadManager` downloads the delta zip with resume support, same as a full OTA zip.
2. **SHA-256 Verification:** The client computes the SHA-256 hash of the downloaded delta zip and compares it against the `sha256` field from the update check response.
3. **Signature Verification:** The RSA-4096 signature of the delta payload is verified against the embedded public key.
4. **Structure Validation:** The ZIP is validated to contain `payload.bin`, `payload_properties.txt`, and `care_map.pb`.

No special delta-specific download or verification logic is needed in the client SDK — the delta is simply a smaller OTA package from the client's perspective.

6.3 Source Partition Read for Delta Application

When applying a delta update, `update_engine` needs to read from the source partition (the inactive slot). This is transparent to the Helix OTA client — it simply calls `update_engine.applyPayload()` with the delta's `payload.bin` URI and offsets, same as with a full update. The `update_engine` daemon handles all source partition reads internally.

However, the client must ensure that the inactive slot contains the expected source version before initiating the delta apply. This is verified by `update_engine` itself through the `source_partition_hash` check in the delta manifest (see Section 2.4).

6.4 Fallback to Full Update on Delta Failure

If the delta application fails, the client must fall back to a full update. The failure detection and fallback flow is:

1. `update_engine` reports a failure via the `onPayloadApplicationComplete` Binder callback.
2. The Helix OTA client's `InstallService` receives the error code.
3. If the error code indicates a delta-specific failure (e.g., `DELTA_PRECHECK_FAILED`, `SOURCE_PARTITION_HASH_MISMATCH`, `DELTA_APPLICATION_FAILED`), the client:
    1. Reports the delta failure to the server via telemetry.
    2. Initiates a full update download using the `full_update_fallback` URL from the update check response.
    3. The full update is downloaded, verified, and applied through the normal update flow.
4. If the error code indicates a generic failure (e.g., network error, storage error), the client retries the delta update using its existing retry logic. Only after the retry budget is exhausted does it fall back to the full update.

The `delta_failed` telemetry event includes detailed error information:

```
{
  "device_id": "dev_01HDEVICE001",
  "event_type": "failure",
  "payload": {
    "stage": "installing",
    "error_code": "DELTA_PRECHECK_FAILED",
    "error_message": "Source partition hash mismatch:
        expected abc123, got def456",
    "artifact_id": "dlt_01HDELTA01",
    "update_type": "DELTA",
    "source_version": "15.0.0",
    "target_version": "15.0.2",
    "fallback_to_full": true
  }
}
```

---

7. New Submodule: helix-delta-gen

7.1 Overview

Property	Value
Repository (GitHub)	`HelixDevelopment/helix-delta-gen`
Repository (GitLab)	`HelixDevelopment/helix-delta-gen`
Go module	`dev.helix.ota.deltagen`
Language	Go 1.22+
License	Apache 2.0
Visibility	Public

`helix-delta-gen` is the delta generation service for the Helix OTA system. It provides both a CLI tool for manual/CI-driven delta generation and a Go library for integration into the server-side delta management pipeline.

7.2 Package Structure

```
helix-delta-gen/
├── cmd/
│   └── helix-delta-gen/
│       └── main.go                        # CLI entry point
├── internal/
│   ├── generator/
│   │   ├── generator.go                   # DeltaGenerator
interface + core impl
│   │   ├── android.go                     # Android
incremental OTA generator
│   │   ├── linux.go                       # Linux rootfs
delta (stub for 1.1.0)
│   │   ├── pipeline.go                    # 6-stage
generation pipeline
│   │   └── generator_test.go
│   ├── extractor/
│   │   ├── payload.go                     # payload.bin
extractor (protobuf parser)
│   │   ├── partition.go                   # Partition image
extractor
│   │   └── extractor_test.go
│   ├── verifier/
│   │   ├── apply.go                       # Delta apply
verification
│   │   ├── hash.go                        # Partition hash
verification
│   │   └── verifier_test.go
│   └── config/
│       ├── config.go                      # Generation
```

```
configuration
│       └── config_test.go
├── pkg/
│   ├── api/
│   │   ├── types.go                    # Public types
(DeltaJob, DeltaResult, etc.)
│   │   └── errors.go                   # Public error
types
│   └── deltagen/
│       ├── service.go                  #
DeltaGenerationService (library API)
│       └── service_test.go
├── scripts/
│   ├── generate_delta_ota.sh           # Shell wrapper
for ota_from_target_files
│   ├── extract_images.py               # Python helper
for partition image extraction
│   ├── build_target_files.py           # Python helper
for target_files structure
│   └── verify_delta.py                 # Python helper
for delta verification
├── proto/
│   └── update_metadata.proto           # Chrome OS
update_engine protobuf defs
├── go.mod
├── go.sum
├── Makefile
├── Dockerfile
├── README.md
├── CLAUDE.md
├── AGENTS.md
└── CONTRIBUTING.md
```

7.3 Core Types and Interfaces

```go
package deltagen

import (
    "context"
    "time"
)

// DeltaGenerator defines the interface for platform-
    specific delta generation.
// Each supported OS type implements this interface.
type DeltaGenerator interface {
    // Generate produces a delta artifact from source to
    target.
    Generate(ctx context.Context, job *DeltaJob)
    (*DeltaResult, error)
```

```
    // Platform returns the OS type this generator supports
        (e.g., "android", "linux").
    Platform() string

    // ValidateInputs checks that the source and target
        artifacts are compatible.
    ValidateInputs(ctx context.Context, source, target
        *ArtifactInfo) error

    // EstimatedDuration returns the expected generation
        time for the given inputs.
    EstimatedDuration(source, target *ArtifactInfo)
        time.Duration

    // RequiredResources returns the compute resources
        needed for generation.
    RequiredResources(source, target *ArtifactInfo)
        *ResourceRequirements
}

// ArtifactInfo contains metadata about a source or target
        OTA artifact.
type ArtifactInfo struct {
    ID                string            `json:"id"`
    Version           string            `json:"version"`
    OSType            string            `json:"os_type"`
    HardwareModels    []string
        `json:"hardware_models"`
    FileSizeBytes     int64
        `json:"file_size_bytes"`
    SHA256            string            `json:"sha256"`
    StorageKey        string
        `json:"storage_key"`
    PartitionMetadata  map[string]int64
        `json:"partition_metadata"` // partition_name →
        size_bytes
}

// DeltaJob represents a single delta generation job.
type DeltaJob struct {
    ID              string         `json:"id"`
    SourceArtifact  *ArtifactInfo  `json:"source_artifact"`
    TargetArtifact  *ArtifactInfo  `json:"target_artifact"`
    Status          DeltaStatus    `json:"status"`
    Priority        int
        `json:"priority"`       // Higher = more urgent
    MaxRetries      int            `json:"max_retries"`
    RetryCount      int            `json:"retry_count"`
    CreatedAt       time.Time      `json:"created_at"`
    StartedAt       *time.Time     `json:"started_at"`
    CompletedAt     *time.Time     `json:"completed_at"`
    Config          DeltaConfig    `json:"config"`
}
```

```
// DeltaStatus represents the lifecycle state of a delta
        generation job.
type DeltaStatus string

const (
    DeltaStatusPending    DeltaStatus = "PENDING"
    DeltaStatusGenerating DeltaStatus = "GENERATING"
    DeltaStatusGenerated  DeltaStatus = "GENERATED"
    DeltaStatusFailed     DeltaStatus = "FAILED"
    DeltaStatusCancelled  DeltaStatus = "CANCELLED"
)

// DeltaResult contains the output of a successful delta
        generation.
type DeltaResult struct {
    JobID              string            `json:"job_id"`
    OutputStorageKey   string
        `json:"output_storage_key"`
    OutputSizeBytes    int64
        `json:"output_size_bytes"`
    OutputSHA256       string
        `json:"output_sha256"`
    SourceSizeBytes    int64
        `json:"source_size_bytes"`
    SavingsPercent     int
        `json:"savings_percent"`
    GenerationTimeSec  float64
        `json:"generation_time_seconds"`
    PartitionDeltas    []PartitionDelta
        `json:"partition_deltas"`
}

// PartitionDelta contains per-partition delta statistics.
type PartitionDelta struct {
    Name             string  `json:"name"`              //
        "system", "vendor", "boot", etc.
    SourceSizeBytes  int64   `json:"source_size_bytes"`
    TargetSizeBytes  int64   `json:"target_size_bytes"`
    DeltaSizeBytes   int64   `json:"delta_size_bytes"`
    SourceCopyOps    int     `json:"source_copy_ops"`   //
        Blocks copied from source
    SourceBsdiffOps  int     `json:"source_bsdiff_ops"` //
        Blocks produced via bsdiff
    PuffdiffOps      int     `json:"puffdiff_ops"`      //
        Blocks produced via puffdiff
    ReplaceOps       int     `json:"replace_ops"`       //
        Blocks with raw replacement
    SavingsPercent   int     `json:"savings_percent"`
}

// DeltaConfig contains configuration for delta generation.
type DeltaConfig struct {
```

```
    WorkerConcurrency int
        `json:"worker_concurrency"` // Parallel partition
        processing
    MaxMemoryGB       int
        `json:"max_memory_gb"`      // Memory limit per job
    TempDir           string
        `json:"temp_dir"`           // Temporary storage
        directory
    SigningKeyPath    string
        `json:"signing_key_path"`   // OTA signing key
    DisablePuffdiff   bool
        `json:"disable_puffdiff"`   // Skip puffdiff (for
        testing)
    DisableVerify     bool
        `json:"disable_verify"`     // Skip post-generation
        verification
    Timeout           time.Duration
        `json:"timeout"`            // Maximum generation
        time
}

// ResourceRequirements describes the compute resources
        needed for a generation job.
type ResourceRequirements struct {
    CPUCores     int   `json:"cpu_cores"`
    MemoryGB     int   `json:"memory_gb"`
    DiskGB       int   `json:"disk_gb"`
    EstimatedSec int64 `json:"estimated_seconds"`
}

// DeltaGenerationService is the main service facade for
        delta generation.
type DeltaGenerationService struct {
    generators map[string]DeltaGenerator // keyed by
        platform name
    storage    StorageProvider
    jobs       JobRepository
    events     EventBusPublisher
    workers    *WorkerPool
}

// NewDeltaGenerationService creates a new service with
        registered generators.
func NewDeltaGenerationService(
    generators []DeltaGenerator,
    storage StorageProvider,
    jobs JobRepository,
    events EventBusPublisher,
    workers *WorkerPool,
) *DeltaGenerationService {
    svc := &DeltaGenerationService{
        generators: make(map[string]DeltaGenerator),
        storage:    storage,
        jobs:       jobs,
```

```
        events:     events,
        workers:    workers,
    }
    for _, g := range generators {
        svc.generators[g.Platform()] = g
    }
    return svc
}

// EnqueueJob creates and enqueues a new delta generation
       job.
func (s *DeltaGenerationService) EnqueueJob(
    ctx context.Context,
    source, target *ArtifactInfo,
    config DeltaConfig,
) (*DeltaJob, error) {
    platform := target.OSType
    gen, ok := s.generators[platform]
    if !ok {
        return nil, fmt.Errorf("no delta generator for
        platform %q", platform)
    }

    if err := gen.ValidateInputs(ctx, source, target); err !
        = nil {
        return nil, fmt.Errorf("input validation: %w", err)
    }

    job := &DeltaJob{
        ID:             generateID("djb_"),
        SourceArtifact: source,
        TargetArtifact: target,
        Status:         DeltaStatusPending,
        Priority:       computePriority(source, target),
        MaxRetries:     3,
        Config:         config,
        CreatedAt:      time.Now(),
    }

    if err := s.jobs.Create(ctx, job); err != nil {
        return nil, fmt.Errorf("create job: %w", err)
    }

    // Enqueue for async processing
    s.workers.Submit(func() {
        s.processJob(context.Background(), job, gen)
    })

    s.events.Publish(ctx, Event{
        Type:    "delta.job.enqueued",
        Payload: job,
    })
```

```
    return job, nil
}

// processJob executes the delta generation pipeline for a
        single job.
func (s *DeltaGenerationService) processJob(
    ctx context.Context,
    job *DeltaJob,
    gen DeltaGenerator,
) {
    now := time.Now()
    job.Status = DeltaStatusGenerating
    job.StartedAt = &now
    s.jobs.Update(ctx, job)

    result, err := gen.Generate(ctx, job)
    if err != nil {
        job.RetryCount++
        if job.RetryCount < job.MaxRetries {
            job.Status = DeltaStatusPending // Re-queue for
        retry
            s.workers.Submit(func() {
                s.processJob(context.Background(), job, gen)
            })
        } else {
            job.Status = DeltaStatusFailed
        }
        completed := time.Now()
        job.CompletedAt = &completed
        s.jobs.Update(ctx, job)

        s.events.Publish(ctx, Event{
            Type:    "delta.job.failed",
            Payload: map[string]interface{}{"job_id":
        job.ID, "error": err.Error()},
        })
        return
    }

    // Upload delta artifact to storage
    if err := s.storage.Upload(ctx,
        result.OutputStorageKey, result.OutputSizeBytes);
        err != nil {
        job.Status = DeltaStatusFailed
        completed := time.Now()
        job.CompletedAt = &completed
        s.jobs.Update(ctx, job)
        return
    }

    job.Status = DeltaStatusGenerated
```

```
    completed := time.Now()
    job.CompletedAt = &completed
    s.jobs.Update(ctx, job)

    s.events.Publish(ctx, Event{
        Type: "delta.generated",
        Payload: map[string]interface{}{
            "job_id":           job.ID,
            "source_version":   job.SourceArtifact.Version,
            "target_version":   job.TargetArtifact.Version,
            "savings_percent":  result.SavingsPercent,
            "delta_size_bytes": result.OutputSizeBytes,
        },
    })
}

// GetJobStatus returns the current status of a delta
        generation job.
func (s *DeltaGenerationService) GetJobStatus(ctx
        context.Context, jobID string) (*DeltaJob, error) {
    return s.jobs.GetByID(ctx, jobID)
}

// ListDeltasForTarget returns all delta artifacts targeting
        a specific version.
func (s *DeltaGenerationService) ListDeltasForTarget(
    ctx context.Context,
    targetArtifactID string,
) ([]*DeltaResult, error) {
    return s.jobs.ListByTarget(ctx, targetArtifactID)
}
```

7.4 Android Delta Generator Implementation

```
package generator

import (
    "context"
    "fmt"
    "os"
    "os/exec"
    "path/filepath"
    "time"

    "dev.helix.ota.deltagen/pkg/deltagen"
)

// AndroidDeltaGenerator generates incremental OTA packages
        for Android A/B devices.
type AndroidDeltaGenerator struct {
    otaToolPath    string // Path to ota_from_target_files
        binary
```

```
    signingKeyPath string // Path to OTA signing key
    pythonPath     string // Path to python3 interpreter
    scriptDir      string // Path to helper scripts
        directory
    tempDir        string // Base directory for temporary
        files
}

// NewAndroidDeltaGenerator creates a new Android delta
        generator.
func NewAndroidDeltaGenerator(cfg AndroidGeneratorConfig)
        *AndroidDeltaGenerator {
    return &AndroidDeltaGenerator{
        otaToolPath:    cfg.OTAToolPath,
        signingKeyPath: cfg.SigningKeyPath,
        pythonPath:     cfg.PythonPath,
        scriptDir:      cfg.ScriptDir,
        tempDir:        cfg.TempDir,
    }
}

func (g *AndroidDeltaGenerator) Platform() string { return
        "android" }

func (g *AndroidDeltaGenerator) ValidateInputs(
    ctx context.Context,
    source, target *deltagen.ArtifactInfo,
) error {
    if source.OSType != "android" || target.OSType !=
        "android" {
        return
        fmt.Errorf("both artifacts must be android OS
        type")
    }
    if source.Version >= target.Version {
        return fmt.Errorf("source version %q must be less
        than target version %q",
            source.Version, target.Version)
    }
    // Check hardware compatibility overlap
    compatible := false
    sourceModels := make(map[string]bool)
    for _, m := range source.HardwareModels {
        sourceModels[m] = true
    }
    for _, m := range target.HardwareModels {
        if sourceModels[m] {
            compatible = true
            break
        }
    }
    if !compatible {
```

```
        return fmt.Errorf("no overlapping hardware models
        between source and target")
    }
    return nil
}

func (g *AndroidDeltaGenerator) EstimatedDuration(
    source, target *deltagen.ArtifactInfo,
) time.Duration {
    // Heuristic: ~1 hour per GB of source partition data
    var totalPartSize int64
    for _, size := range source.PartitionMetadata {
        totalPartSize += size
    }
    hours := totalPartSize / (1 << 30) // GB
    if hours < 2 {
        hours = 2
    }
    return time.Duration(hours) * time.Hour
}

func (g *AndroidDeltaGenerator) RequiredResources(
    source, target *deltagen.ArtifactInfo,
) *deltagen.ResourceRequirements {
    var totalPartSize int64
    for _, size := range source.PartitionMetadata {
        totalPartSize += size
    }
    gb := totalPartSize / (1 << 30)
    return &deltagen.ResourceRequirements{
        CPUCores:     8,
        MemoryGB:     int(gb * 8), // bsdiff needs ~8x
        partition size in RAM
        DiskGB:       int(gb * 10), // source + target +
        delta + temp
        EstimatedSec: int64(g.Seconds()),
    }
}

// Generate produces an incremental OTA package.
func (g *AndroidDeltaGenerator) Generate(
    ctx context.Context,
    job *deltagen.DeltaJob,
) (*deltagen.DeltaResult, error) {
    workDir := filepath.Join(g.tempDir, job.ID)
    if err := os.MkdirAll(workDir, 0755); err != nil {
        return nil, fmt.Errorf("create work dir: %w", err)
    }
    defer os.RemoveAll(workDir)

    outputZip := filepath.Join(workDir, "delta_ota.zip")
```

```
    // Execute the delta generation script
    scriptPath := filepath.Join(g.scriptDir,
        "generate_delta_ota.sh")
    cmd := exec.CommandContext(ctx, "/bin/bash", scriptPath,
        job.SourceArtifact.StorageKey,
        job.TargetArtifact.StorageKey,
        outputZip,
        g.signingKeyPath,
    )
    cmd.Dir = workDir
    cmd.Stdout = os.Stdout
    cmd.Stderr = os.Stderr

    start := time.Now()
    if err := cmd.Run(); err != nil {
        return nil, fmt.Errorf("delta generation script
        failed: %w", err)
    }
    generationTime := time.Since(start)

    // Read output file stats
    stat, err := os.Stat(outputZip)
    if err != nil {
        return nil, fmt.Errorf("stat output file: %w", err)
    }

    deltaSize := stat.Size()
    sourceSize := job.SourceArtifact.FileSizeBytes
    savings := 100 - int((deltaSize*100)/sourceSize)

    // Compute SHA-256 of the delta
    sha256, err := computeFileSHA256(outputZip)
    if err != nil {
        return nil, fmt.Errorf("compute delta sha256: %w",
        err)
    }

    return &deltagen.DeltaResult{
        JobID:             job.ID,
        OutputStorageKey:  fmt.Sprintf("deltas/%s/%s/
        delta_ota.zip",
            job.SourceArtifact.ID, job.TargetArtifact.ID),
        OutputSizeBytes:   deltaSize,
        OutputSHA256:      sha256,
        SourceSizeBytes:   sourceSize,
        SavingsPercent:    savings,
        GenerationTimeSec: generationTime.Seconds(),
    }, nil
}

func computeFileSHA256(path string) (string, error) {
    f, err := os.Open(path)
```

```
    if err != nil {
        return "", err
    }
    defer f.Close()
    h := sha256.New()
    if _, err := io.Copy(h, f); err != nil {
        return "", err
    }
    return hex.EncodeToString(h.Sum(nil)), nil
}
```

7.5 CLI Tool

The `helix-delta-gen` CLI tool provides manual delta generation for CI/CD pipelines and one-off operations:

```
# Generate a delta between two OTA zips
helix-delta-gen generate \
    --source-ota /path/to/source_ota.zip \
    --target-ota /path/to/target_ota.zip \
    --output /path/to/delta_ota.zip \
    --signing-key /path/to/release_key.pem \
    --platform android \
    --verify

# Check generation status
helix-delta-gen status --job-id djb_01HJOB001

# List available deltas for a target version
helix-delta-gen list --target-version 15.0.2

# Clean up old delta artifacts
helix-delta-gen cleanup --max-source-age 90d --max-per-
        target 5
```

---

8. Performance Benchmarks

8.1 Expected Bandwidth Savings (60–90%)

Based on AOSP's delta generation statistics and real-world Android OTA deployments, the expected bandwidth savings for the Orange Pi 5 Max (RK3588) are:

Update Type	Full OTA Size	Delta OTA Size	Savings	Notes
Patch-level	2.1 GB	150–300 MB	85–93%	

Update Type	Full OTA Size	Delta OTA Size	Savings	Notes
(15.0.0 → 15.0.1)				Small changes; most blocks identical
Minor version (15.0.x → 15.1.0)	2.3 GB	400–800 MB	65–83%	More changed blocks; framework updates
Major version (14.x → 15.0.0)	2.5 GB	1.0–1.5 GB	40–60%	Significant partition changes; fewer SOURCE_COPY ops

The 60–90% bandwidth reduction target in the Version Roadmap (Section 6.2) is achievable for typical minor and patch-level updates, which represent the vast majority of production OTA cycles. Major version updates may fall below 60% but are rare (typically annual).

8.2 Delta Generation Time Estimates

Source→Target	Partition Data	Generation Time	Peak RAM	Notes
Patch-level (15.0.0→15.0.1)	4 GB/slot	2–3 hours	32 GB	Mostly SOURCE_COPY; fast bsdiff
Minor version (15.0.2→15.1.0)	4 GB/slot	3–5 hours	32 GB	More bsdiff/ puffdiff operations
Major version (14.x→15.0.0)	4 GB/slot	5–8 hours	32 GB	Heavy puffdiff; few SOURCE_COPY ops

Generation time is dominated by the puffdiff computation, which is CPU-intensive and requires decompression of both source and target partitions. Parallelizing by partition (system, vendor, boot, product, odm processed concurrently) reduces wall-clock time by 2–3× on an 8-core machine.

8.3 Delta Application Time vs. Full Update

Update Type	Download Time (50 Mbps)	Apply Time	Total Update Time	Full Update Total
Patch-level delta	~30 sec	3–5 min	~5.5 min	~10 min
Minor version delta	~2 min	5–8 min	~10 min	~10 min
Full update	~5.5 min	5–8 min	~13.5 min	~13.5 min

Delta application time is often comparable to full update application time because the delta apply involves reading from the source partition (additional I/O) and computing bsdiff/puffdiff patches (additional CPU). The primary time saving is in download, not apply.

For minor version deltas, the total update time may be only marginally faster than a full update. The primary value proposition is bandwidth savings, not time savings, for minor version updates. For patch-level deltas, both bandwidth and time savings are significant.

8.4 Storage Overhead for Delta Artifacts

Scenario	Full Artifacts	Delta Artifacts	Total Storage	Delta Overhead
5 versions, 3 source deltas each	5 × 2.1 GB = 10.5 GB	15 × ~300 MB = 4.5 GB	15.0 GB	+43%
10 versions, 3 source deltas each	10 × 2.1 GB = 21.0 GB	30 × ~300 MB = 9.0 GB	30.0 GB	+43%
20 versions, 5 source deltas each	20 × 2.1 GB = 42.0 GB	100 × ~300 MB = 30.0 GB	72.0 GB	+71%

With the default cleanup policy (max 3 source deltas per target, max 90-day source age), the storage overhead is approximately 43%. This is a reasonable trade-off given the 60–90% bandwidth savings on every update cycle.

---

9. Reference Implementation: Go Delta Management Service

The following Go code implements the server-side delta management service that integrates with the existing Helix OTA server's service layer:

```
package service

import (
    "context"
    "fmt"
    "time"

    "dev.helix.ota.deltagen/pkg/deltagen"
    "digital.vasic/cache"
    "digital.vasic/eventbus"
)

// DeltaService manages the delta update lifecycle on the
        server side.
// It is responsible for: triggering delta generation,
        selecting the optimal
// delta for a device during update checks, tracking delta
        generation status,
// and enforcing delta cleanup policies.
type DeltaService struct {
    deltaRepo    DeltaArtifactRepository
    artifactRepo ArtifactRepository
    genService   *deltagen.DeltaGenerationService
    storage      StorageProvider
    cache        cache.Provider
    events       eventbus.Publisher
    maxSources   int           // Max source versions per
        target (default: 3)
    maxSourceAge
        time.Duration // Max age of source artifact for
        delta gen (default: 90 days)
}

// NewDeltaService creates a new DeltaService with the given
        dependencies.
func NewDeltaService(
    deltaRepo DeltaArtifactRepository,
    artifactRepo ArtifactRepository,
```

```
    genService *deltagen.DeltaGenerationService,
    storage StorageProvider,
    cache cache.Provider,
    events eventbus.Publisher,
) *DeltaService {
    return &DeltaService{
        deltaRepo:    deltaRepo,
        artifactRepo: artifactRepo,
        genService:   genService,
        storage:      storage,
        cache:        cache,
        events:       events,
        maxSources:   3,
        maxSourceAge: 90 * 24 * time.Hour,
    }
}

// OnArtifactUploaded is called when a new artifact is
        uploaded.
// It triggers delta generation for the N most recent source
        versions.
func (s *DeltaService) OnArtifactUploaded(ctx
        context.Context, targetArtifactID string) error {
    target, err := s.artifactRepo.GetByID(ctx,
        targetArtifactID)
    if err != nil {
        return fmt.Errorf("lookup target artifact: %w", err)
    }

    // Find eligible source artifacts (same OS type,
        compatible hardware, recent)
    sources, err :=
        s.artifactRepo.FindEligibleDeltaSources(ctx,
        &EligibleSourceFilter{
        OSType:            target.OSType,
        HardwareModels:    target.HardwareCompatibility,
        ExcludeArtifactID: targetArtifactID,
        MaxAge:            s.maxSourceAge,
        Limit:             s.maxSources,
    })
    if err != nil {
        return fmt.Errorf("find eligible sources: %w", err)
    }

    for _, source := range sources {
        // Skip if delta already exists
        existing, _ := s.deltaRepo.GetBySourceTarget(ctx,
        source.ID, target.ID)
        if existing != nil {
            continue
        }

        // Create pending delta record
```

```
        delta := &DeltaArtifact{
            ID:                generateID("dlt_"),
            SourceArtifactID:  source.ID,
            TargetArtifactID:  target.ID,
            SourceVersion:     source.Version,
            TargetVersion:     target.Version,
            OSType:            target.OSType,
            HardwareModel:
        target.HardwareCompatibility[0],
            Status:            "PENDING",
            CreatedAt:         time.Now(),
        }
        if err := s.deltaRepo.Create(ctx, delta); err !=
        nil {
            return fmt.Errorf("create delta record: %w",
        err)
        }

        // Enqueue generation job
        sourceInfo := artifactToDeltaInfo(source)
        targetInfo := artifactToDeltaInfo(target)

        job, err := s.genService.EnqueueJob(ctx,
        sourceInfo, targetInfo, deltagen.DeltaConfig{
            WorkerConcurrency: 1,
            MaxMemoryGB:       32,
            SigningKeyPath:    "/etc/helix/
        ota_signing_key.pem",
            Timeout:           8 * time.Hour,
        })
        if err != nil {
            delta.Status = "FAILED"
            s.deltaRepo.Update(ctx, delta)
            continue
        }

        delta.GenerationJobID = job.ID
        delta.Status = "GENERATING"
        s.deltaRepo.Update(ctx, delta)
    }

    return nil
}

// SelectDeltaForDevice determines the best delta for a
        device during an update check.
// Returns nil if no delta is available (caller should fall
        back to full update).
func (s *DeltaService) SelectDeltaForDevice(
    ctx context.Context,
    deviceSourceVersion string,
    targetArtifactID string,
    hardwareModel string,
```

```
) (*DeltaArtifact, error) {
    // Try cache first
    cacheKey := fmt.Sprintf("delta:%s:%s:%s",
        deviceSourceVersion, targetArtifactID,
        hardwareModel)
    if cached, err := s.cache.Get(ctx, cacheKey); err ==
        nil {
        return cached.(*DeltaArtifact), nil
    }

    // Query for matching delta
    delta, err := s.deltaRepo.GetBySourceVersionTarget(
        ctx, deviceSourceVersion, targetArtifactID,
        hardwareModel,
    )
    if err != nil {
        return nil, fmt.Errorf("lookup delta: %w", err)
    }

    if delta == nil || delta.Status != "GENERATED" {
        return nil, nil // No delta available
    }

    // Cache for 5 minutes
    s.cache.Set(ctx, cacheKey, delta, 5*time.Minute)

    return delta, nil
}

// ComputeDeltaChainLength determines the number of
        sequential deltas needed
// to update from sourceVersion to targetVersion. Returns
        the chain length,
// or -1 if no valid chain exists.
func (s *DeltaService) ComputeDeltaChainLength(
    ctx context.Context,
    sourceVersion string,
    targetVersion string,
    hardwareModel string,
) (int, error) {
    if sourceVersion == targetVersion {
        return 0, nil
    }

    // BFS through the delta graph
    visited := make(map[string]bool)
    queue := []struct {
        version string
        depth   int
    }{{version: sourceVersion, depth: 0}}

    for len(queue) > 0 {
        current := queue[0]
```

```
        queue = queue[1:]

        if visited[current.version] {
            continue
        }
        visited[current.version] = true

        if current.version == targetVersion {
            return current.depth, nil
        }

        // Find all deltas from current version
        deltas, err := s.deltaRepo.ListBySourceVersion(ctx,
        current.version, hardwareModel)
        if err != nil {
            return -1, fmt.Errorf("list deltas from %s:
        %w", current.version, err)
        }

        for _, d := range deltas {
            if d.Status == "GENERATED" && !
        visited[d.TargetVersion] {
                queue = append(queue, struct {
                    version string
                    depth   int
                }{version: d.TargetVersion, depth:
        current.depth + 1})
            }
        }
    }

    return -1, nil // No path found
}

// CleanupOldDeltas removes delta artifacts that exceed the
        retention policy.
func (s *DeltaService) CleanupOldDeltas(ctx
        context.Context) (int, error) {
    cleaned := 0

    // 1. Remove deltas with source artifact older than
        maxSourceAge
    expired, err := s.deltaRepo.ListExpiredSources(ctx,
        s.maxSourceAge)
    if err != nil {
        return 0, fmt.Errorf("list expired deltas: %w", err)
    }
    for _, delta := range expired {
        if err := s.deleteDelta(ctx, delta); err != nil {
            continue
        }
        cleaned++
    }
```

```
    // 2. Enforce max-per-target limit (keep only maxSources
        most recent)
    targets, err := s.deltaRepo.ListDistinctTargets(ctx)
    if err != nil {
        return cleaned, fmt.Errorf("list targets: %w", err)
    }
    for _, targetID := range targets {
        deltas, err := s.deltaRepo.ListByTarget(ctx,
        targetID)
        if err != nil {
            continue
        }
        if len(deltas) > s.maxSources {
            // Sort by source artifact creation date (oldest
        first)
            // Delete the oldest ones exceeding the limit
            for i := 0; i < len(deltas)-s.maxSources; i++ {
                s.deleteDelta(ctx, deltas[i])
                cleaned++
            }
        }
    }

    // 3. Remove failed deltas older than 7 days
    failed, err := s.deltaRepo.ListFailedOlderThan(ctx,
        7*24*time.Hour)
    if err != nil {
        return cleaned, fmt.Errorf("list failed deltas:
        %w", err)
    }
    for _, delta := range failed {
        s.deleteDelta(ctx, delta)
        cleaned++
    }

    s.events.Publish(ctx, eventbus.Event{
        Type:    "delta.cleaned_up",
        Payload: map[string]int{"cleaned_count": cleaned},
    })

    return cleaned, nil
}

func (s *DeltaService) deleteDelta(ctx context.Context,
        delta *DeltaArtifact) error {
    // Delete from object storage
    if delta.StorageKey != "" {
        s.storage.Delete(ctx, delta.StorageKey)
    }
    // Soft-delete from database
    return s.deltaRepo.Delete(ctx, delta.ID)
}
```

```go
// DeltaArtifact represents a delta OTA package in the
        database.
type DeltaArtifact struct {
    ID                string    `json:"id" db:"id"`
    SourceArtifactID  string    `json:"source_artifact_id"
        db:"source_artifact_id"`
    TargetArtifactID  string    `json:"target_artifact_id"
        db:"target_artifact_id"`
    SourceVersion     string    `json:"source_version"
        db:"source_version"`
    TargetVersion     string    `json:"target_version"
        db:"target_version"`
    OSType            string    `json:"os_type"
        db:"os_type"`
    HardwareModel     string    `json:"hardware_model"
        db:"hardware_model"`
    StorageKey        string    `json:"storage_key"
        db:"storage_key"`
    FileSizeBytes     int64     `json:"file_size_bytes"
        db:"file_size_bytes"`
    SHA256            string    `json:"sha256" db:"sha256"`
    SavingsPercent    int       `json:"savings_percent"
        db:"savings_percent"`
    GenerationJobID   string    `json:"generation_job_id"
        db:"generation_job_id"`
    GenerationTimeSec float64   `json:"generation_time_sec"
        db:"generation_time_sec"`
    Status            string    `json:"status" db:"status"`
    CreatedAt         time.Time `json:"created_at"
        db:"created_at"`
    UpdatedAt         time.Time `json:"updated_at"
        db:"updated_at"`
}

// DeltaArtifactRepository defines the data access interface
        for delta artifacts.
type DeltaArtifactRepository interface {
    Create(ctx context.Context, delta *DeltaArtifact) error
    Update(ctx context.Context, delta *DeltaArtifact) error
    Delete(ctx context.Context, id string) error
    GetByID(ctx context.Context, id string)
        (*DeltaArtifact, error)
    GetBySourceTarget(ctx context.Context, sourceID,
        targetID string) (*DeltaArtifact, error)
    GetBySourceVersionTarget(ctx context.Context,
        sourceVersion, targetID, hardwareModel string)
        (*DeltaArtifact, error)
    ListBySourceVersion(ctx context.Context, sourceVersion,
        hardwareModel string) ([]*DeltaArtifact, error)
    ListByTarget(ctx context.Context, targetID string)
        ([]*DeltaArtifact, error)
    ListDistinctTargets(ctx context.Context) ([]string,
        error)
    ListExpiredSources(ctx context.Context, maxAge
        time.Duration) ([]*DeltaArtifact, error)
```

```
    ListFailedOlderThan(ctx context.Context, age
        time.Duration) ([]*DeltaArtifact, error)
}
```

---

10. Reference Implementation: Android Delta Generation Shell Scripts

10.1 Delta Generation Wrapper Script

```
#!/bin/bash
# generate_delta_ota.sh — Generate an Android incremental
        OTA package
#
# This script wraps the AOSP ota_from_target_files tool to
        produce
# delta OTA packages from two full OTA zip files.
#
# Prerequisites:
#   - Android build tools (ota_from_target_files,
        brillo_update_payload)
#   - Python 3 with protobuf library
#   - Minimum 32 GB RAM, 40 GB free disk space
#
# Usage:
#   generate_delta_ota.sh <source_ota.zip> <target_ota.zip>
        <output.zip> [signing_key]
#
# Environment Variables:
#   HELIX_DELTA_WORK_DIR   - Base directory for temporary
        files (default: /tmp/helix_delta)
#   HELIX_DELTA_CONCURRENCY - Number of parallel partition
        diff jobs (default: 4)
#   HELIX_DELTA_TIMEOUT    - Maximum generation time in
        seconds (default: 28800 = 8h)

set -euo pipefail

# ============================================================================
# Configuration
# ============================================================================

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
WORK_BASE="${HELIX_DELTA_WORK_DIR:-/tmp/helix_delta}"
CONCURRENCY="${HELIX_DELTA_CONCURRENCY:-4}"
TIMEOUT="${HELIX_DELTA_TIMEOUT:-28800}"
LOG_PREFIX="[helix-delta-gen]"

# ============================================================================
```

```
# Input Validation
# ================================================================================

if [[ $# -lt 3 ]]; then
    echo "Usage: $0 <source_ota.zip> <target_ota.zip>
        <output.zip> [signing_key]"
    exit 1
fi

SOURCE_OTA="$(realpath "$1")"
TARGET_OTA="$(realpath "$2")"
OUTPUT_ZIP="$(realpath "$3")"
SIGNING_KEY="${4:-}"

for f in "${SOURCE_OTA}" "${TARGET_OTA}"; do
    if [[ ! -f "${f}" ]]; then
        echo "${LOG_PREFIX} ERROR: File not found: ${f}" >&2
        exit 1
    fi
done

# Verify ZIP structure
for f in "${SOURCE_OTA}" "${TARGET_OTA}"; do
    if ! unzip -l "${f}" | grep -q "payload.bin"; then
        echo "${LOG_PREFIX} ERROR: Missing payload.bin in $
        {f}" >&2
        exit 1
    fi
    if ! unzip -l "${f}" | grep -q
        "payload_properties.txt"; then
        echo "${LOG_PREFIX} ERROR: Missing
        payload_properties.txt in ${f}" >&2
        exit 1
    fi
done

# ================================================================================
# Prepare Working Directory
# ================================================================================

TIMESTAMP=$(date +%Y%m%d_%H%M%S)
WORK_DIR="${WORK_BASE}/delta_${TIMESTAMP}_$$"
mkdir -p "$
        {WORK_DIR}/"{source,target,source_images,target_images,delta_output}

cleanup() {
    echo "${LOG_PREFIX} Cleaning up work directory: $
        {WORK_DIR}"
    rm -rf "${WORK_DIR}"
}
trap cleanup EXIT
```

```
echo "${LOG_PREFIX}
        ============================================="
echo "${LOG_PREFIX} Helix OTA Delta Generation"
echo "${LOG_PREFIX}
        ============================================="
echo "${LOG_PREFIX} Source: ${SOURCE_OTA} ($(numfmt --
        to=iec $(stat -c%s "${SOURCE_OTA}")))"
echo "${LOG_PREFIX} Target: ${TARGET_OTA} ($(numfmt --
        to=iec $(stat -c%s "${TARGET_OTA}")))"
echo "${LOG_PREFIX} Output: ${OUTPUT_ZIP}"
echo "${LOG_PREFIX} Work Dir: ${WORK_DIR}"
echo "${LOG_PREFIX} Concurrency: ${CONCURRENCY}"
echo "${LOG_PREFIX} Timeout: ${TIMEOUT}s"

# ================================================================================
# Stage 1: Extract payload.bin from both OTA zips
# ================================================================================

echo "${LOG_PREFIX} [Stage 1/6] Extracting payloads..."

unzip -q -o "${SOURCE_OTA}" "payload.bin"
        "payload_properties.txt" \
    -d "${WORK_DIR}/source/"
unzip -q -o "${TARGET_OTA}" "payload.bin"
        "payload_properties.txt" \
    -d "${WORK_DIR}/target/"

# Read source and target partition info from
        payload_properties
SOURCE_PROPERTIES="${WORK_DIR}/source/
        payload_properties.txt"
TARGET_PROPERTIES="${WORK_DIR}/target/
        payload_properties.txt"

echo "${LOG_PREFIX} Source payload properties:"
cat "${SOURCE_PROPERTIES}"
echo "${LOG_PREFIX} Target payload properties:"
cat "${TARGET_PROPERTIES}"

# ================================================================================
# Stage 2: Extract partition images
# ================================================================================

echo "${LOG_PREFIX} [Stage 2/6] Extracting partition
        images..."

PARTITIONS="system vendor boot product odm"

# Extract source partition images
python3 "${SCRIPT_DIR}/extract_images.py" \
    --payload "${WORK_DIR}/source/payload.bin" \
    --output-dir "${WORK_DIR}/source_images/" \
    --partitions ${PARTITIONS} \
```

```
    --concurrency "${CONCURRENCY}"

# Extract target partition images
python3 "${SCRIPT_DIR}/extract_images.py" \
    --payload "${WORK_DIR}/target/payload.bin" \
    --output-dir "${WORK_DIR}/target_images/" \
    --partitions ${PARTITIONS} \
    --concurrency "${CONCURRENCY}"

# Verify extraction
for part in ${PARTITIONS}; do
    if [[ -f "${WORK_DIR}/source_images/${part}.img" ]];
        then
        SRC_SIZE=$(stat -c%s "${WORK_DIR}/source_images/$
        {part}.img")
        echo "${LOG_PREFIX}   Source ${part}.img: $(numfmt
        --to=iec ${SRC_SIZE})"
    fi
    if [[ -f "${WORK_DIR}/target_images/${part}.img" ]];
        then
        TGT_SIZE=$(stat -c%s "${WORK_DIR}/target_images/$
        {part}.img")
        echo "${LOG_PREFIX}   Target ${part}.img: $(numfmt
        --to=iec ${TGT_SIZE})"
    fi
done

# ================================================================================
# Stage 3: Build target_files-like structures
# ================================================================================

echo "${LOG_PREFIX} [Stage 3/6] Building target_files
        structures..."

python3 "${SCRIPT_DIR}/build_target_files.py" \
    --source-images "${WORK_DIR}/source_images/" \
    --target-images "${WORK_DIR}/target_images/" \
    --source-ota "${SOURCE_OTA}" \
    --target-ota "${TARGET_OTA}" \
    --output-dir "${WORK_DIR}/target_files/"

# ================================================================================
# Stage 4: Compute delta diff using ota_from_target_files
# ================================================================================

echo "${LOG_PREFIX} [Stage 4/6] Computing delta diff (this
        may take several hours)..."

START_TIME=$(date +%s)

OTA_CMD="ota_from_target_files"
OTA_ARGS=(
    --incremental_from "${WORK_DIR}/target_files/source/"
```

```
    --worker_threads "${CONCURRENCY}"
)

if [[ -n "${SIGNING_KEY}" ]]; then
    OTA_ARGS+=(-k "${SIGNING_KEY}")
fi

OTA_ARGS+=(
    "${WORK_DIR}/target_files/target/"
    "${WORK_DIR}/delta_output/delta_ota.zip"
)

# Run with timeout
timeout "${TIMEOUT}" ${OTA_CMD} "${OTA_ARGS[@]}" 2>&1 | \
    while IFS= read -r line; do
        echo "${LOG_PREFIX}   ${line}"
    done

OTA_EXIT=${PIPESTATUS[0]}
END_TIME=$(date +%s)
ELAPSED=$((END_TIME - START_TIME))

if [[ ${OTA_EXIT} -ne 0 ]]; then
    echo "${LOG_PREFIX} ERROR: ota_from_target_files failed
        with exit code ${OTA_EXIT}" >&2
    exit ${OTA_EXIT}
fi

echo "${LOG_PREFIX} Delta diff computation completed in $
        {ELAPSED} seconds"

# ================================================================================
# Stage 5: Verify delta package
# ================================================================================

echo "${LOG_PREFIX} [Stage 5/6] Verifying delta package..."

python3 "${SCRIPT_DIR}/verify_delta.py" \
    --delta "${WORK_DIR}/delta_output/delta_ota.zip" \
    --source-images "${WORK_DIR}/source_images/" \
    --target-images "${WORK_DIR}/target_images/" \
    --partitions ${PARTITIONS}

echo "${LOG_PREFIX} Delta verification passed"

# ================================================================================
# Stage 6: Copy to output location
# ================================================================================

echo "${LOG_PREFIX} [Stage 6/6] Copying to output
        location..."
```

```
cp "${WORK_DIR}/delta_output/delta_ota.zip" "${OUTPUT_ZIP}"

# ================================================================================
# Summary
# ================================================================================

DELTA_SIZE=$(stat -c%s "${OUTPUT_ZIP}")
SOURCE_SIZE=$(stat -c%s "${SOURCE_OTA}")
TARGET_SIZE=$(stat -c%s "${TARGET_OTA}")
SAVINGS_VS_SOURCE=$((100 - (DELTA_SIZE * 100 /
        SOURCE_SIZE)))
SAVINGS_VS_TARGET=$((100 - (DELTA_SIZE * 100 /
        TARGET_SIZE)))

echo "${LOG_PREFIX}
        ============================================="
echo "${LOG_PREFIX} Delta Generation Complete"
echo "${LOG_PREFIX}
        ============================================="
echo "${LOG_PREFIX} Delta size:    $(numfmt --to=iec $
        {DELTA_SIZE})"
echo "${LOG_PREFIX} Source OTA:    $(numfmt --to=iec $
        {SOURCE_SIZE})"
echo "${LOG_PREFIX} Target OTA:    $(numfmt --to=iec $
        {TARGET_SIZE})"
echo "${LOG_PREFIX} Savings (vs source): $
        {SAVINGS_VS_SOURCE}%"
echo "${LOG_PREFIX} Savings (vs target): $
        {SAVINGS_VS_TARGET}%"
echo "${LOG_PREFIX} Generation time: ${ELAPSED}s"
echo "${LOG_PREFIX} Output: ${OUTPUT_ZIP}"
echo "${LOG_PREFIX}
        ============================================="
```

10.2 Delta Application Verification Script

```
#!/bin/bash
# verify_delta_apply.sh — Verify a delta OTA by applying it
        to source images
# and comparing the result with target images.
#
# Usage: verify_delta_apply.sh <delta_ota.zip>
        <source_images_dir> <target_images_dir>
#
# This script extracts the delta payload, applies it to
        source partition images,
# and compares SHA-256 hashes of the resulting images with
        the target images.

set -euo pipefail

DELTA_ZIP="$1"
SOURCE_DIR="$2"
TARGET_DIR="$3"
```

```
LOG_PREFIX="[helix-delta-verify]"

WORK_DIR=$(mktemp -d /tmp/helix_verify_XXXXXX)
trap "rm -rf ${WORK_DIR}" EXIT

echo "${LOG_PREFIX} Verifying delta: ${DELTA_ZIP}"

# Extract delta payload
unzip -q -o "${DELTA_ZIP}" "payload.bin"
        "payload_properties.txt" -d "${WORK_DIR}/"

# Parse partition names and expected hashes from the delta
        manifest
python3 "$(dirname "$0")/parse_manifest.py" \
    --payload "${WORK_DIR}/payload.bin" \
    --output-json "${WORK_DIR}/manifest.json"

# For each partition in the manifest:
PARTITIONS=$(python3 -c "import json; m=json.load(open('$
        {WORK_DIR}/manifest.json')); print('
        '.join(m['partitions'].keys()))")

for part in ${PARTITIONS}; do
    SOURCE_IMG="${SOURCE_DIR}/${part}.img"
    TARGET_IMG="${TARGET_DIR}/${part}.img"

    if [[ ! -f "${SOURCE_IMG}" ]]; then
        echo "${LOG_PREFIX} WARNING: Source image not found
        for ${part}, skipping"
        continue
    fi

    if [[ ! -f "${TARGET_IMG}" ]]; then
        echo "${LOG_PREFIX} WARNING: Target image not found
        for ${part}, skipping"
        continue
    fi

    # Compute source partition hash
    SOURCE_HASH=$(sha256sum "${SOURCE_IMG}" | cut -d' ' -f1)
    TARGET_HASH=$(sha256sum "${TARGET_IMG}" | cut -d' ' -f1)

    # Get expected hashes from manifest
    EXPECTED_SOURCE=$(python3 -c "import json;
        m=json.load(open('${WORK_DIR}/manifest.json'));
        print(m['partitions']['${part}']['source_hash'])")
    EXPECTED_TARGET=$(python3 -c "import json;
        m=json.load(open('${WORK_DIR}/manifest.json'));
        print(m['partitions']['${part}']['target_hash'])")

    # Verify source hash
    if [[ "${SOURCE_HASH}" != "${EXPECTED_SOURCE}" ]]; then
```

```
        echo "${LOG_PREFIX} ERROR: Source hash mismatch for
            ${part}" >&2
        echo "${LOG_PREFIX}   Expected: ${EXPECTED_SOURCE}"
            >&2
        echo "${LOG_PREFIX}   Actual:   ${SOURCE_HASH}" >&2
        exit 1
    fi
    echo "${LOG_PREFIX} Source hash verified for ${part}: $
        {SOURCE_HASH:0:16}..."

    # Verify target hash
    if [[ "${TARGET_HASH}" != "${EXPECTED_TARGET}" ]]; then
        echo "${LOG_PREFIX} ERROR: Target hash mismatch for
            ${part}" >&2
        echo "${LOG_PREFIX}   Expected: ${EXPECTED_TARGET}"
            >&2
        echo "${LOG_PREFIX}   Actual:   ${TARGET_HASH}" >&2
        exit 1
    fi
    echo "${LOG_PREFIX} Target hash verified for ${part}: $
        {TARGET_HASH:0:16}..."
done

echo "${LOG_PREFIX} All partition hashes verified
        successfully"
```

---

11. Database Schema Additions

11.1 `delta_artifacts` Table

```
-- Delta artifacts: incremental OTA packages generated from
        source→target version pairs
CREATE TABLE delta_artifacts (
    id                  UUID        PRIMARY KEY DEFAULT
        gen_random_uuid(),
    source_artifact_id  UUID        NOT NULL REFERENCES
        artifacts(id) ON DELETE CASCADE,
    target_artifact_id  UUID        NOT NULL REFERENCES
        artifacts(id) ON DELETE CASCADE,
    source_version      VARCHAR(64) NOT NULL,
    target_version      VARCHAR(64) NOT NULL,
    os_type             VARCHAR(64) NOT NULL,
    hardware_model      VARCHAR(128) NOT NULL,
    storage_key         VARCHAR(1024),
    file_size_bytes     BIGINT      DEFAULT 0,
    file_hash_sha256    VARCHAR(128),
    source_size_bytes   BIGINT      DEFAULT 0,
    savings_percent     INTEGER     DEFAULT 0,
    generation_job_id   VARCHAR(64),
    generation_time_sec FLOAT       DEFAULT 0,
```

```
    status              VARCHAR(32) NOT NULL DEFAULT
        'PENDING',
    partition_deltas    JSONB       DEFAULT '[]',
    created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    deleted_at          TIMESTAMPTZ,

    CONSTRAINT chk_delta_source_lt_target CHECK
        (source_version < target_version),
    CONSTRAINT chk_delta_file_size_positive CHECK
        (file_size_bytes >= 0),
    CONSTRAINT chk_delta_savings_range CHECK
        (savings_percent >= 0 AND savings_percent <= 100),
    CONSTRAINT chk_delta_status CHECK (status IN
        ('PENDING', 'GENERATING', 'GENERATED', 'FAILED',
        'CANCELLED')),
    CONSTRAINT uidx_delta_source_target UNIQUE
        (source_artifact_id, target_artifact_id) WHERE
        deleted_at IS NULL
);

COMMENT ON TABLE delta_artifacts IS
        'Delta (incremental) OTA packages containing binary
        diffs between source and target versions';
COMMENT ON COLUMN delta_artifacts.source_artifact_id IS
        'The full OTA artifact representing the source
        (current) version';
COMMENT ON COLUMN delta_artifacts.target_artifact_id IS
        'The full OTA artifact representing the target
        (new) version';
COMMENT ON COLUMN delta_artifacts.savings_percent IS
        'Bandwidth savings percentage compared to the
        source full OTA artifact';
COMMENT ON COLUMN delta_artifacts.partition_deltas IS 'Per-partition delta stat
        [{"name":"system","source_copy_ops":500,"source_bsdiff_ops":20,"puffdi
COMMENT ON COLUMN delta_artifacts.generation_job_id IS 'ID
        of the delta generation job in helix-delta-gen';
```

11.2 Indexes for delta_artifacts

```
-- Find delta by source→target pair (most common query in
        update check)
CREATE INDEX idx_delta_source_target ON delta_artifacts
        (source_artifact_id, target_artifact_id)
    WHERE deleted_at IS NULL;

-- Find deltas by source version string (for delta chain
        computation)
CREATE INDEX idx_delta_source_version ON delta_artifacts
        (source_version, os_type, hardware_model)
    WHERE deleted_at IS NULL AND status = 'GENERATED';

-- Find deltas by target artifact (for listing available
        deltas)
```

```
CREATE INDEX idx_delta_target ON delta_artifacts
        (target_artifact_id, status)
    WHERE deleted_at IS NULL;

-- Find deltas by status (for generation monitoring)
CREATE INDEX idx_delta_status ON delta_artifacts (status,
        created_at DESC)
    WHERE deleted_at IS NULL;

-- Find expired deltas for cleanup
CREATE INDEX idx_delta_cleanup ON delta_artifacts
        (source_artifact_id, status, updated_at)
    WHERE deleted_at IS NULL AND status IN ('GENERATED',
        'FAILED');

-- Find deltas by OS type and hardware model (for
        compatibility queries)
CREATE INDEX idx_delta_compat ON delta_artifacts (os_type,
        hardware_model, source_version, target_version)
    WHERE deleted_at IS NULL AND status = 'GENERATED';
```

11.3 Artifact Metadata Schema Update

The `artifacts` table's `payload_metadata` JSONB column is extended with delta-related fields:

```
{
    "update_type": "full",
    "source_version_min": "15.0.0",
    "partitions": {
        "system": 2147483648,
        "vendor": 536870912,
        "boot": 67108864,
        "product": 419430400,
        "odm": 209715200
    },
    "partition_hashes": {
        "system": "sha256:abc123...",
        "vendor": "sha256:def456...",
        "boot": "sha256:789abc...",
        "product": "sha256:012def...",
        "odm": "sha256:345678..."
    },
    "care_map_present": true,
    "ab_ota": true,
    "delta_eligible": true
}
```

The `partition_hashes` and `delta_eligible` fields are new in 1.0.2. The `delta_eligible` flag indicates whether this artifact's source images can be used for delta generation.

Artifacts with `delta_eligible: false` (e.g., those with unknown partition hashes or cross-grade updates) will never have deltas generated from them.

---

12. API Additions for Delta Updates

12.1 Extended Update Check Response

The `POST /api/v1/updates/check` response is extended with the following fields when a delta is available:

Field	Type	Description
`update_type`	string	`"FULL"` or `"DELTA"`
`source_version`	string	Source version for the delta (matches device's current version)
`size_savings_percent`	integer	Bandwidth savings percentage
`full_update_fallback`	object	Full update artifact info for fallback

12.2 Delta Generation Status Endpoint

`GET /api/v1/artifacts/{artifact_id}/deltas`

Returns all delta artifacts targeting the specified artifact.

Authentication: Bearer JWT (admin, operator, or viewer role)

Response:

```
{
  "data": [
    {
      "id": "dlt_01HDELTA01",
      "source_artifact_id": "art_01HSOURCE",
      "source_version": "15.0.0",
      "target_artifact_id": "art_01HTARGET",
      "target_version": "15.0.2",
      "os_type": "android",
      "hardware_model": "rk3588_opi5max",
      "file_size_bytes": 245000000,
      "source_size_bytes": 2100000000,
      "savings_percent": 88,
      "status": "GENERATED",
      "generation_time_sec": 7200,
```

```
      "created_at": "2026-03-05T10:00:00Z"
    }
  ],
  "pagination": {
    "total_count": 3,
    "has_more": false,
    "next_cursor": "",
    "limit": 50
  }
}
```

12.3 Manual Delta Generation Trigger

`POST /api/v1/deltas/generate`

Manually trigger delta generation for a specific source→target pair. This is useful when automatic generation failed or was not triggered.

Authentication: Bearer JWT (admin or operator role)

Request Body:

```
{
  "source_artifact_id": "art_01HSOURCE",
  "target_artifact_id": "art_01HTARGET"
}
```

Response (202 Accepted):

```
{
  "job_id": "djb_01HJOB001",
  "source_artifact_id": "art_01HSOURCE",
  "target_artifact_id": "art_01HTARGET",
  "status": "PENDING",
  "estimated_duration_seconds": 7200,
  "created_at": "2026-03-05T10:00:00Z"
}
```

12.4 Delta Generation Job Status

`GET /api/v1/deltas/jobs/{job_id}`

Response:

```
{
  "job_id": "djb_01HJOB001",
  "source_version": "15.0.0",
  "target_version": "15.0.2",
  "status": "GENERATING",
  "progress_percent": 45,
  "started_at": "2026-03-05T10:05:00Z",
```

```
  "estimated_completion_at": "2026-03-05T12:05:00Z",
  "created_at": "2026-03-05T10:00:00Z"
}
```

---

13. Risk Assessment

Risk	Probability	Impact	Mitigation
Delta generation takes too long (>8h for large partitions)	Medium	Medium	Parallelize by partition; limit delta to consecutive versions; set hard timeout with retry
Delta apply fails on device due to source partition drift	Medium	High	Source partition hash verification before delta apply; automatic fallback to full update on mismatch
Storage overhead: N×M delta combinations	Medium	Medium	Only generate deltas for last N source versions; enforce max-per-target limit; automated cleanup
bsdiff memory usage during generation (>16 GB RAM for 2 GB partition)	Low	High	Use streaming bsdiff variant; run on dedicated high-memory builder node; cap memory at 32 GB per job
Delta generation fails silently, no delta available for rollout	Low	High	Monitoring on generation job failures; alerting if no GENERATED delta exists for a target artifact within 6 hours of upload; manual trigger API
Race condition:	Low	Medium	Delta selection only returns

Risk	Probability	Impact	Mitigation
device checks for update while delta is being regenerated			deltas with status GENERATED; in-progress deltas are not served
Delta package signing key differs from full OTA signing key	Low	Critical	Use the same signing key for delta and full OTAs; key rotation must generate new deltas
puffdiff not available on build server	Medium	Medium	Fallback to SOURCE_BSDIFF only; document reduced savings; provide configuration flag to disable puffdiff

14. Testing Requirements

Test Category	Specific Tests	Tool
Unit	Delta selection logic, source→target mapping, chain length computation, cleanup policy, fallback trigger	Go `testing`, testify
Mutation	85% mutation score on delta service, generator, and selection code	`go-mutesting`
Integration	Upload artifact → delta generated → device checks → receives delta → applies → reports; delta failure → full update fallback	`testcontainers-go`
E2E	Full delta update cycle on real	

Test Category	Specific Tests	Tool
	Orange Pi 5 Max hardware; measure actual bandwidth reduction vs. full update	Custom hardware test harness
Performance	Delta generation time benchmarked at ≤ 4h for 2 GB partition; delta apply time ≤ 1.2× full update apply time	Custom benchmarks
Security	Verify source partition hash check prevents tampered delta apply; verify delta signing; verify fallback on hash mismatch	OWASP ZAP, custom pen-test scripts
Chaos	Kill delta generation job mid-computation; verify job retries; kill update_engine during delta apply; verify fallback	Custom chaos scripts

14.1 Test Matrix for Delta Selection

Source Version	Target Version	Delta Exists	Expected Result
15.0.0	15.0.1	Yes (GENERATED)	Return delta
15.0.0	15.0.1	Yes (GENERATING)	Return full update
15.0.0	15.0.1	Yes (FAILED)	Return full update
15.0.0	15.0.1	No	Return full update
15.0.0	15.0.2	Direct delta exists	Return direct delta

Source Version	Target Version	Delta Exists	Expected Result
15.0.0	15.0.3	Chain 15.0.0→15.0.1→15.0.2→15.0.3 (length 3)	Return chain info, serve first delta
15.0.0	15.0.4	Chain length 4	Return full update
Unknown	15.0.1	No source artifact	Return full update

End of Document — HELOTA-DELTA-001 v1.0.2